Identifying the LLM Behind the Text: CNN and LSTM Classifiers for LLM Fingerprinting

Team leader: Siddhant Bhagat (solo)

Repository: https://github.com/Sid-Bhagat/llm-fingerprinting

1. Problem Definition

Given a response written by a large language model, we predict which LLM family produced it. As more of the text people read online comes from models like GPT, Claude, PaLM and Llama, it becomes useful to know whether these models leave consistent stylistic fingerprints that a classifier can pick up.

We frame this as a four class text classification problem over the families GPT, Claude, Llama and PaLM. Each example is a triple of (LLM name, user prompt, LLM response), and we train two required architectures, a convolutional neural network (TextCNN) and a recurrent neural network (bidirectional LSTM).

This report addresses two research questions:

- **RQ1:** Can we identify which LLM family generated a response, using only the response text?
- **RQ2:** Does the user's prompt help? We compare classifiers trained on the prompt only, the response only, and the prompt and response together.

2. Dataset Curation

Our final dataset has 9,192 responses, balanced at roughly 2,300 per family, drawn from the public Chatbot Arena conversations dataset. We chose it because every row already contains a model name, a user prompt and a model response, and because each prompt is answered by two randomly assigned models, which matters for RQ2.

Source

We used lmsys/chatbot_arena_conversations on Hugging Face (arXiv:2306.05685). It is a gated dataset under a CC license, and access was granted after accepting its terms. The raw data has 33,000 rows covering 20 models, where each row is one user prompt answered by two different models in a blind comparison.

Family mapping

We grouped models by the organization or base model behind them. Every model outside these four families was dropped, since each comes from a different base model (RWKV-4-Raven-14B, chatglm-6b, stablelm-tuned-alpha-7b, dolly-v2-12b, oasst-pythia-12b, mpt-7b-chat, fastchat-t5-3b).

Family	Models included
GPT	gpt-4, gpt-3.5-turbo
Claude	claude-v1, claude-instant-v1
Llama	vicuna-13b, vicuna-7b, koala-13b, alpaca-13b, wizardlm-13b, gpt4all-13b-snoozy, guanaco-33b
PaLM	palm-2

The Llama class is made up of community fine tunes of the original LLaMA rather than Meta's own chat model. Several of them, including Vicuna, were fine tuned on ChatGPT conversations, so we expected some confusion between the GPT and Llama classes from the start.

Filtering pipeline

1. Keep single turn, English conversations (25,938 rows, or 51,876 responses).
2. Split each row into two examples, one per model, with the user turn as the input and the assistant turn as the output.
3. Map models to families and drop the rest, then drop empty outputs and exact duplicate (input, output) pairs. This leaves GPT 6,944, Claude 5,142, Llama 16,895 and PaLM 2,305.
4. Remove self identification. Models sometimes name themselves ("I am Vicuna, trained by LMSYS"), which would let a classifier cheat. We dropped any response mentioning a model or company name such as Claude, Anthropic, OpenAI, ChatGPT, Vicuna, LMSYS, PaLM, Bard or Google, or containing the phrase "trained by". We dropped these responses instead of masking the names, because a placeholder token would itself become a signal if one family self identifies more often.
5. Label each prompt's domain as code, math or general using keyword rules.
6. Split 70/15/15 by question ID, so the same prompt never appears in two splits.
7. Balance each split by downsampling every family to that split's smallest family.

The self identification filter removed very little, which tells us it was not a major leakage source in this data:

Family	Responses dropped for self identification
GPT	22
Claude	28
Llama	35
PaLM	7

We split before balancing on purpose. Balancing first and then splitting by question ID can leave the splits unbalanced, while splitting first lets every split be exactly balanced. The final splits are 6,332 training, 1,416 validation and 1,444 test examples, with no question ID shared between splits.

Domain labels

Domain labels come from simple rules on the prompt. Code is checked first, using code fences and code keywords. A prompt is labeled math if it contains clear math terms (equation, integral, prime, divisible, solve for, percent), an arithmetic expression such as 2+2 with dates removed first, a "how many" or "how much" question with at least two numbers, or is mostly digits and operators. Everything else is general.

Family	Code	General	Math
GPT	291	1,847	160
Claude	290	1,848	160
Llama	298	1,841	159
PaLM	321	1,794	183

Limitations

- PaLM is the smallest family, so it sets the class size and caps the dataset at about 9,200 examples.

- Domain labels are heuristic. In a spot check of 20 prompts labeled math, about 16 were real math problems, and known false positives include "Compare 9/11 and January 6th."
- All models date from 2023, so results may not carry over to current models.

3. Models and Training

We trained a TextCNN and a bidirectional LSTM in PyTorch, using the same tokenizer, vocabulary, embedding size and training procedure for both so that differences in results come from the architectures themselves.

Tokenization

We tokenize text into words and individual punctuation marks, and we keep case, punctuation and line breaks, with each newline mapped to a `<nl>` token. Most text classification pipelines lowercase and strip punctuation, but here formatting is part of the signal we want to detect, since models differ in how they use lists, line breaks and headers. The vocabulary is built from the training split only, capped at the 30,000 most frequent tokens, and saved once so every run shares it.

Sequence lengths depend on the input setting. Input only uses the first 64 prompt tokens, output only uses the first 256 response tokens, and the combined setting uses 64 prompt tokens, a `<sep>` token and 192 response tokens.

Architectures

	TextCNN	BiLSTM
Embedding	128 dimensions, learned	128 dimensions, learned
Core layer	Convolutions with filter widths 3, 4 and 5, 100 filters each	One bidirectional LSTM layer, hidden size 128 per direction
Pooling	Max over time for each filter	Max over time across hidden states
Dropout	0.5	0.5
Output	Linear layer to 4 classes	Linear layer to 4 classes

The TextCNN slides filters of width 3 to 5 across the token sequence, so each filter learns to respond to a short phrase, and max pooling keeps the strongest match anywhere in the text. The BiLSTM reads the sequence in both directions and pools over all hidden states,

which in principle lets it capture longer range structure. In both models, padding positions are excluded from the max pool, so the amount of padding in a batch cannot change a prediction. The TextCNN has 3,995,104 trainable parameters and the BiLSTM has 4,105,220. In both, 3,840,000 of these (about 96%) are the embedding table, so the models differ mainly in their much smaller core layers, with 155,104 parameters for the convolutions and classifier against 265,220 for the LSTM and classifier.

Training setup

- **Optimizer:** Adam, learning rate 0.001, batch size 64, cross entropy loss.
- **Stopping:** up to 10 epochs, with early stopping on validation macro F1 and a patience of 2. We evaluate the checkpoint from the best validation epoch on the test set.
- **Seeds:** every configuration is trained with seeds 0, 1 and 2, and we report the mean and standard deviation.
- **Metric:** macro F1 is our main metric, alongside accuracy. Because the classes are balanced, the two are close, and chance level for both is 0.25.
- **Hardware:** all runs used a laptop CPU. The full grid of 18 runs (2 models, 3 input settings, 3 seeds) took about an hour, with the LSTM on full responses taking most of that time.

Before the full grid, we ran a one epoch smoke test on 20% of the training data for every configuration to confirm that the loss decreased and that nothing crashed. It also gave the runtime estimate we used to decide that a GPU was not needed.

4. Results

Both models identify the LLM family from the response alone at about 2.7 times chance, with the TextCNN reaching 0.688 macro F1, while the prompt alone carries no usable signal.

Input setting	TextCNN macro F1	TextCNN accuracy	BiLSTM macro F1	BiLSTM accuracy
Output only	**0.688 ± 0.005**	0.685 ± 0.006	0.646 ± 0.060	0.643 ± 0.058
Input + output	0.675 ± 0.012	0.672 ± 0.013	0.648 ± 0.010	0.650 ± 0.007
Input only	0.244 ± 0.012	0.261 ± 0.008	0.252 ± 0.009	0.259 ± 0.005

Values are the mean ± standard deviation over seeds 0, 1 and 2 on the test set of 1,444 responses. Chance level is 0.25. Figure 1 shows the same results as a bar chart.

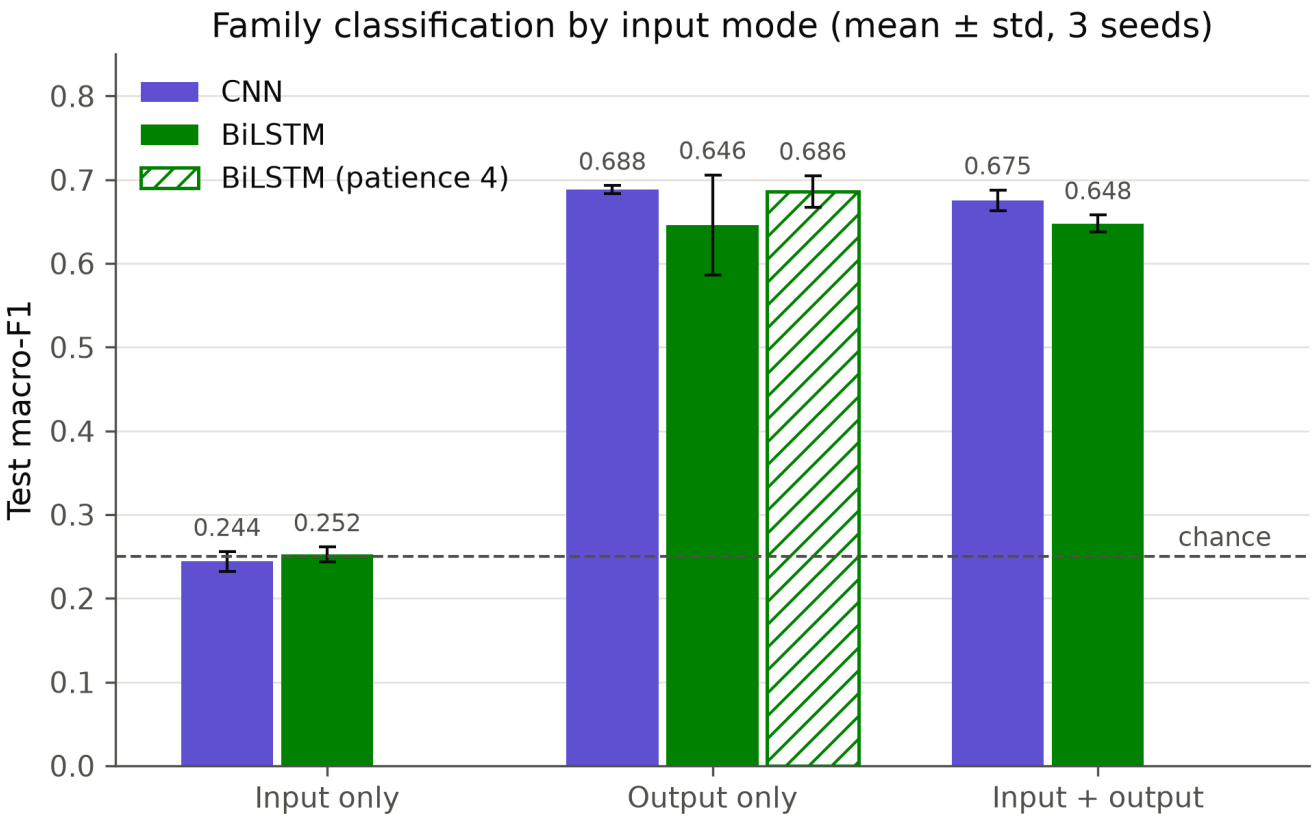


Figure 1. Test macro F1 by input setting, mean ± standard deviation over three seeds. The dashed line marks chance (0.25).

RQ1: Can we identify which LLM generated a response?

Yes. Using only the response, the TextCNN reaches 0.688 macro F1 and the BiLSTM reaches 0.646, both far above the 0.25 chance level.

The TextCNN is more reliable, but the BiLSTM's lower mean comes mostly from one unlucky training run rather than a weaker architecture. Its three seeds scored 0.664, 0.694 and 0.579 macro F1. Seeds 0 and 1 kept improving until epochs 8 and 7, and seed 1 actually beat the TextCNN's mean. Seed 2 peaked at epoch 2, dipped at epoch 3, and was stopped at epoch 4 before it could recover, because with a patience of 2 a single noisy epoch is enough to end training.

To test this explanation, we reran the three BiLSTM seeds with a patience of 4. Seed 0 produced an identical result, since it had already run all 10 epochs. Seed 1 trained one epoch longer and finished almost unchanged, moving from 0.694 to 0.693. Seed 2 kept training and rose from 0.579 to 0.700, the best of all six BiLSTM runs. The first epochs of every rerun matched the original runs exactly, so training is reproducible for a given seed. The BiLSTM mean went from 0.646 ± 0.060 to 0.686 ± 0.019, essentially tied with the TextCNN's 0.688 ± 0.005. We keep patience 2 as the main result, because it is the setting we chose before seeing any results, and report patience 4 as a sensitivity check. The conclusion is that the two architectures reach about the same accuracy, but the TextCNN gets there more consistently and is far less sensitive to the stopping rule. Figure 2 shows the training loss and validation curves for seed 2 under both settings, which are identical until patience 2 stops the run.

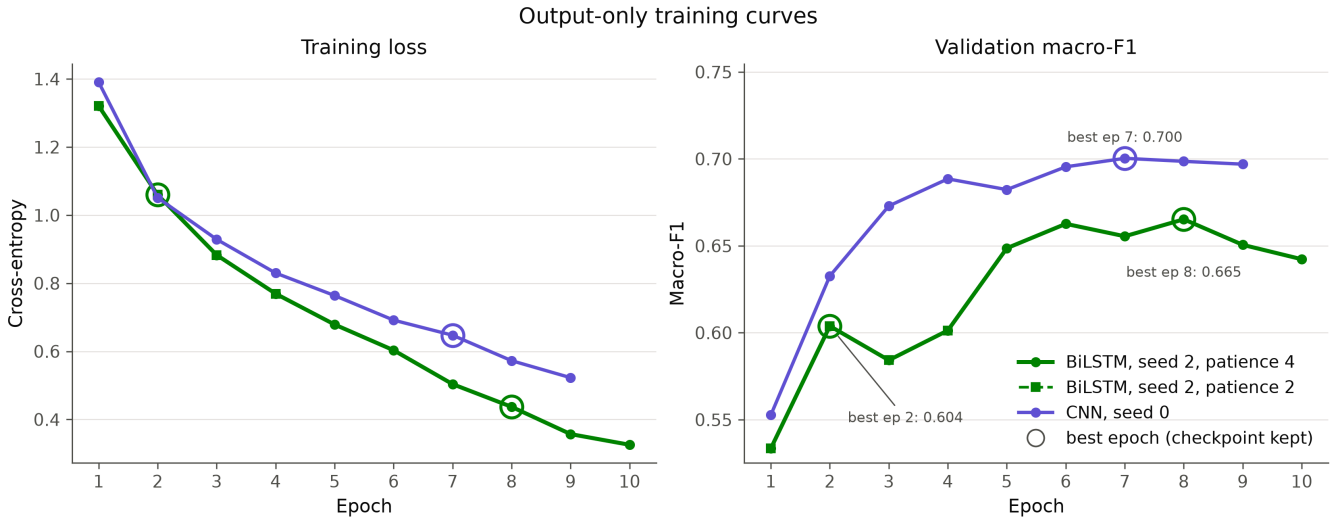


Figure 2. Training loss and validation macro F1 per epoch for the TextCNN (seed 0) and the BiLSTM (seed 2) with patience 2 and 4. Rings mark the best epoch, whose checkpoint is kept. The two BiLSTM runs are identical until patience 2 stops at epoch 4.

The TextCNN, by contrast, converges quickly and consistently, with a standard deviation of only 0.005 across seeds. Our explanation is that the strongest cues are local. Opening phrases, stock expressions and formatting tokens are short n grams, which a CNN's width

3 to 5 filters detect directly, while the LSTM has to learn to carry that information across 256 steps, which takes more epochs and makes its training noisier on about 6,300 examples. We test the idea that the cues are local in the RQ4 analysis.

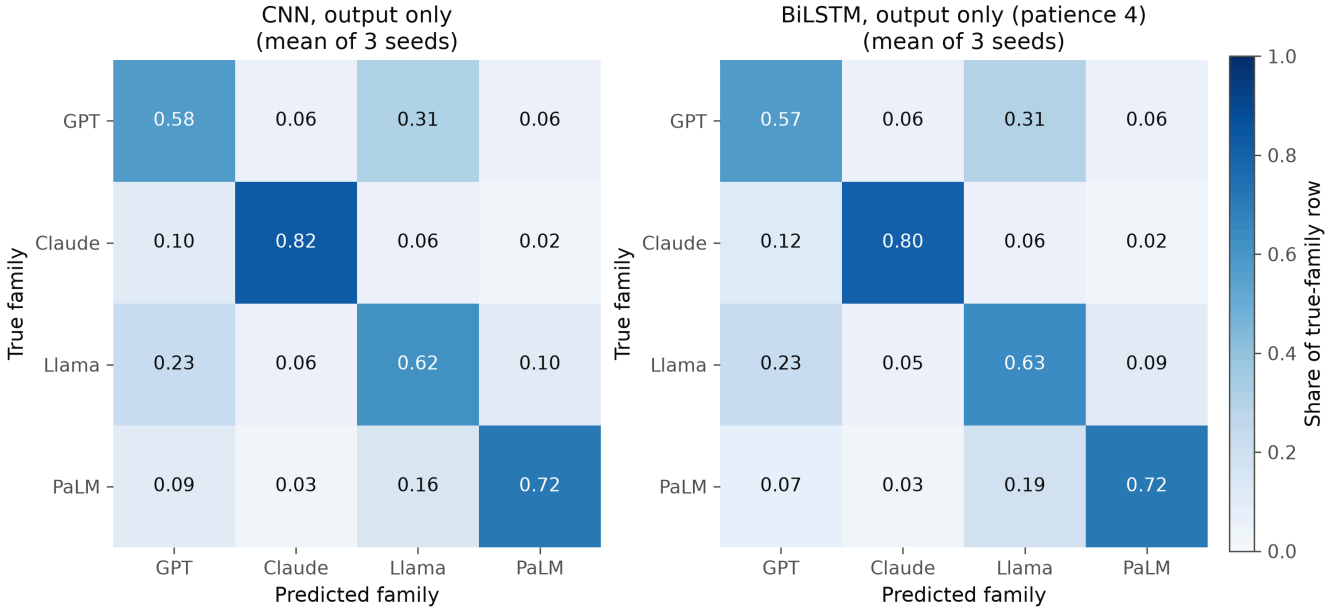


Figure 3. Row normalized confusion matrices on the test set, averaged over three seeds, for the TextCNN (left) and the BiLSTM with patience 4 (right). Each row shows how responses from one true family were classified.

The confusion matrices in Figure 3 show that the families are not equally easy to identify. Averaged over the three seeds, TextCNN recall is 0.83 for Claude, 0.72 for PaLM, 0.62 for Llama and only 0.58 for GPT. GPT's largest error is being labeled as Llama, which happens for 31% of GPT responses, and the reverse error is the second largest, with 23% of Llama responses labeled as GPT. This matches what we expected from the data, since several of the Llama fine tunes were trained on text generated by OpenAI models, for example Alpaca on text-davinci-003 outputs and Vicuna on shared ChatGPT conversations, and learned to imitate that style. Section 5 tests this explanation model by model. In effect, the Llama class partly contains second hand GPT writing, so the boundary between the two is blurry. The BiLSTM shows almost exactly the same pattern, with GPT recall of 0.57, 31% of GPT responses labeled as Llama and 23% of Llama responses labeled as GPT. Two different architectures making the same mistakes suggests the confusion comes from the data rather than from either model.

RQ2: Does the user's prompt help identify the LLM?

Can the LLM be predicted from the prompt alone? No. Both models score at chance from the prompt alone, with 0.244 macro F1 for the TextCNN and 0.252 for the BiLSTM. During these runs the training loss kept falling, from about 1.40 to 1.20, while validation F1 stayed near 0.26. The models were memorizing individual training prompts, and that memorization did not carry over to new prompts, so early stopping kept an early epoch.

Why does input only fail here? This is a property of how the data was collected. In Chatbot Arena, a user writes a prompt and two models are assigned to answer it at random, so the prompt is independent of which model replies. With the prompts split by question ID, nothing in a test prompt can point to a family. In a dataset where people pick a model for a task, for example sending coding questions to GPT 4, input only classification could beat chance by learning the task distribution rather than any fingerprint, so a good input only result should be read as a sign of selection bias in the data rather than a real fingerprint.

Does including the prompt improve classification? No. Adding the prompt lowered TextCNN macro F1 slightly, from 0.688 to 0.675, and left the BiLSTM unchanged at 0.648 compared with 0.646. However, this comparison has a confound. In the combined setting the response is cut to 192 tokens to make room for the prompt, compared with 256 tokens in the output only setting, so the drop could come from seeing less of the response rather than from the prompt.

To separate the two effects, we trained the TextCNN on output only with responses cut to 192 tokens, which scored 0.681 ± 0.010. About half of the 0.013 drop therefore comes from the shorter response, and the remaining 0.006 attributed to the prompt is smaller than the standard deviation across seeds. The prompt neither helps nor meaningfully hurts, which is what we expect when it carries no information about the family.

5. In Depth Analyses

Comparison with simple baselines

A linear model on TF-IDF n grams matches both neural models. To see how much the CNN and LSTM add over simpler methods, we trained logistic regression on TF-IDF features of the response text, once with word unigrams and bigrams and once with character 3 to 5 grams (50,000 features each, case kept, regularization tuned on validation).

Model	Test macro F1
TF-IDF character 3 to 5 grams + logistic regression	0.692
TF-IDF word 1 to 2 grams + logistic regression	0.691
TextCNN	0.688 ± 0.005
BiLSTM, patience 4	0.686 ± 0.019
BiLSTM, patience 2	0.646 ± 0.060
12 handcrafted features + logistic regression	0.435

The TF-IDF models are deterministic, so they are single runs without a standard deviation.

The two TF-IDF variants, the TextCNN and the patience 4 BiLSTM all land within 0.006 of each other, and their per family results match closely too. The character model scores F1 of 0.83 for Claude, 0.76 for PaLM, 0.59 for Llama and 0.58 for GPT, with GPT and Llama again confused with each other more than any other pair. With about 6,300 training examples, the neural models do not find anything that a bag of short word or character sequences cannot. This fits the rest of our results. If the fingerprint lives in short surface patterns, a CNN's narrow filters learn essentially the same features as an n gram model, and the LSTM's ability to model longer range structure has nothing extra to use.

The highest weighted character n grams for each family point to a cue we had not measured directly, which is the character each family uses for bullet points:

Family	Top character n grams
GPT	a newline followed by "- " (hyphen bullets), "As "
Claude	" • " (a Unicode bullet character), "Here "
PaLM	a newline followed by "* " (asterisk bullets), "**" (bold)
Llama	code fences (```), " and "

Each family formats lists with a different bullet character, which a classifier can pick up from three characters of text, and the opening words "As" and "Here" match the opening phrases found in RQ4. The bullet character is an especially sharp cue for Claude. 540 of Claude's 2,298 responses (23.5%) contain •, compared with 2 or fewer for each of the

other families. The strip markdown ablation in RQ4 removes bullets only when they start a line and are followed by a space, so 37 of those 540 Claude responses (about 7%, and 4 of 80 in the test set) still contained a • after stripping. The ablation therefore removed most but not all of this cue, and its 0.036 drop slightly understates how much formatting contributes.

RQ4: What characteristics distinguish different LLMs?

The fingerprint is both structural and linguistic, but most of it lives in the wording, and especially in how a response begins. Formatting and length carry a real but smaller share, and the signal is redundant enough that removing any one part costs between 4 and 7 points of macro F1.

Structural and length features

We computed 12 handcrafted features for all 9,192 responses and ranked them with a Kruskal Wallis test of how strongly each differs across families. Bold use, newline rate, list use and sentence count differ most, while header use and question marks barely differ at all.

Feature (mean)	GPT	Claude	Llama	PaLM
Words (median)	92	186	102	169
Type token ratio, first 100 words	0.718	0.684	0.669	0.642
Words per sentence	14.2	11.2	16.3	13.4
Newlines per 100 words	6.3	9.7	5.6	7.4
Share of lines that are list items	0.18	0.36	0.16	0.21
Responses with any bold text	1.2% or less	1.2% or less	1.2% or less	13.3%

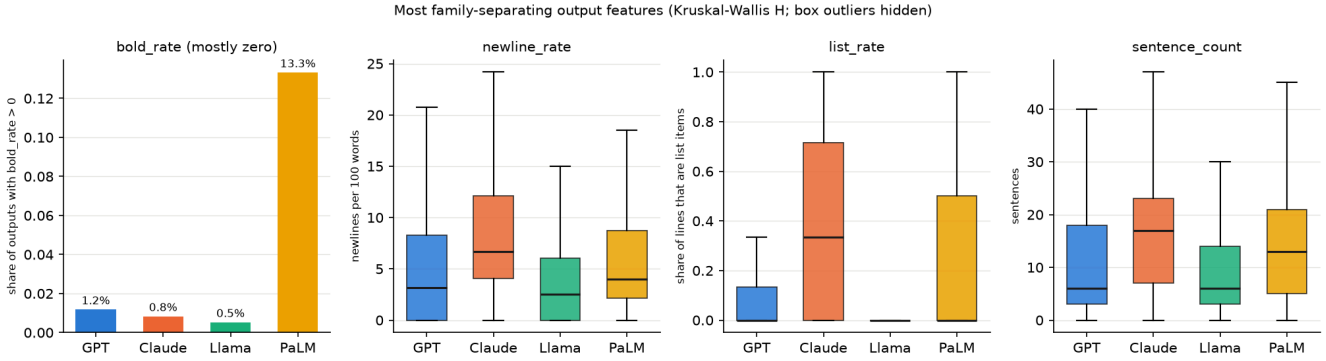


Figure 4. The four features that differ most across families by Kruskal Wallis H. Bold use is shown as the share of responses with any bold text, since most responses have none.

Each family has a recognizable structural habit. Claude writes the longest, most list heavy responses, with the shortest sentences. PaLM is the only family that regularly uses bold text, in 13.3% of its responses compared with about 1% or less for the others. GPT has the highest lexical diversity and the shortest typical response, while Llama writes the longest sentences. GPT and Llama are the closest pair on almost every feature, which matches the confusion between them in RQ1.

A logistic regression trained on only these 12 features reaches 0.435 macro F1, well above the 0.25 chance level but far below the TextCNN's 0.688. Its weakest class is again GPT, with an F1 of 0.304. Structure and length therefore explain part of the fingerprint, and the remaining gap has to come from the words themselves.

How responses open

Family	Most common opening phrases (share of responses)
Claude	"Here is a" (13.5%), "Here are some" (9.2%), "I apologize, but" (5.7%)
PaLM	"I'm not able" (5.0%), "Sure, here is" (4.7%)
GPT	"As an AI" (3.6%), "I'm sorry, but" (2.4%)
Llama	"I'm sorry, but" (2.6%), "As an AI" (1.9%)

Openings are the clearest single signal. Claude starts more than one in eight responses with "Here is a", and its refusals start with "I apologize" rather than "I'm sorry". PaLM refuses with "I'm not able". GPT and Llama share the same top openings, "As an AI" and "I'm sorry, but", which again points to Llama models having learned ChatGPT's phrasing. These phrases do not name a model, so they are not the kind of leakage our filter removed. They are habitual phrasing, which is exactly the fingerprint this project is about. Because these cues are three word phrases near the start of the text, they also support our explanation from RQ1 that the TextCNN's short filters are well suited to this task.

Removing signals

To test how much each kind of signal matters, we retrained both models on output only text with formatting removed (code fences, headers, bullets, bold and italic markers, and repeated blank lines) and, separately, with each response cut to its first 100 tokens.

Setting	TextCNN macro F1	BiLSTM macro F1
Full response, 256 tokens	0.688 ± 0.005	0.646 ± 0.060
Markdown stripped	0.652 ± 0.018	0.650 ± 0.016
First 100 tokens only	0.640 ± 0.032	0.629 ± 0.007

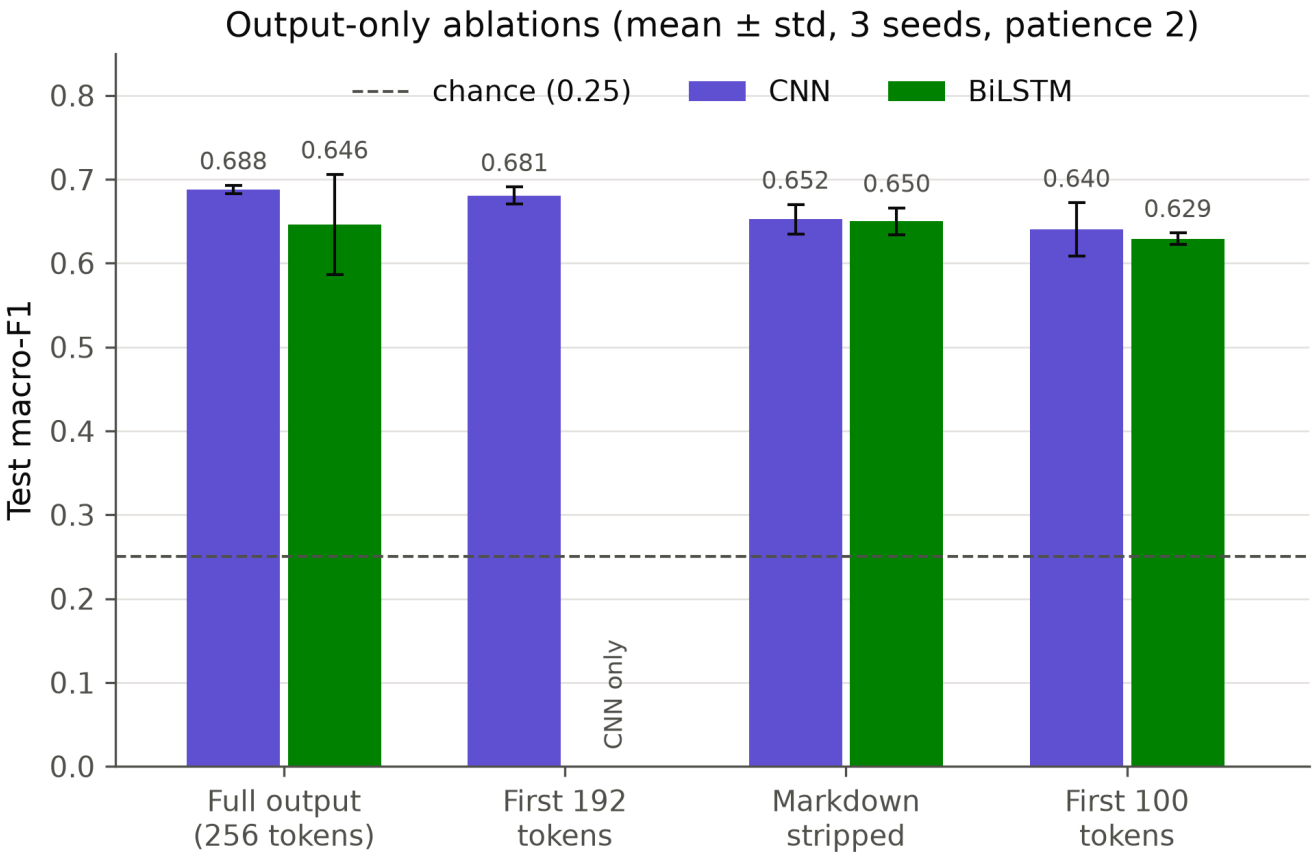


Figure 5. Output only ablations, mean ± standard deviation over three seeds, all with patience 2. The 192 token control was run for the TextCNN only.

Figure 5 shows these results as a chart. Stripping markdown lowers the TextCNN by 0.036, so formatting contributes, but the model still reaches 0.652 without it. This is a striking gap compared with the feature model. The structural features alone are worth 0.435, yet removing them from the text costs less than 0.04, which means the network

can find most of the same information in the wording. Cutting responses to 100 tokens lowers the TextCNN by 0.048, which also removes most of the length signal, since nearly all responses longer than 100 tokens now look the same. Even so, the first 100 tokens alone give 0.640, so the fingerprint is concentrated near the start of a response. To test the role of openings directly, we also retrained the TextCNN with the first sentence of every response removed. This produced the largest drop of any ablation, from 0.688 to 0.622 ± 0.021, even though it removes far less text than truncation does. About 8% of test responses (116 of 1,444) were left unchanged, because they had no sentence boundary or nothing after the first sentence, so the true effect of the opening is if anything slightly larger. A single opening sentence carries more identifying signal than all of the markdown formatting in a response. The BiLSTM comparisons are noisier, because its patience 2 baseline includes the early stopped seed from RQ1, but its scores under both ablations fall in the same range.

To answer the RQ4 questions directly, the classifier appears to rely mostly on wording, especially opening and refusal phrases, with formatting and length as secondary cues. The signals are both linguistic and structural, and they overlap heavily. Removing the opening sentence hurts most (a drop of 0.066), more than removing formatting (0.036) or everything after the first 100 tokens (0.048). Even so, no single removal brings performance anywhere near chance, so no single signal is responsible for the fingerprint.

RQ3: Do LLM fingerprints generalize across domains?

The fingerprint partly generalizes. Trained without any code prompts, both models still identify the family of code answers at more than twice chance, but they lose about 13 points of macro F1 compared with domains they were trained on.

Setup

We trained both models on output only text from general and math prompts, then tested them two ways. The in domain test uses the general and math prompts from the regular test split (1,249 responses). The cross domain test uses every code prompt in the dataset, pooled from all three splits and balanced to 290 responses per family (1,160 responses). Pooling is safe because no code example is used for training or early stopping in these runs, and it gives a test set about six times larger than the code portion of the test split alone. Both conditions use exactly the same trained models, so any difference comes from the test domain and not from the amount of training data. The TextCNN used the default patience of 2, and the BiLSTM used a patience of 4 because RQ1 showed patience 2 makes it unstable.

Results

Model	Test domain	Macro F1	GPT	Claude	Llama	PaLM
TextCNN	General + math (seen)	0.689 ± 0.024	0.544	0.850	0.600	0.763
TextCNN	Code (unseen)	0.563 ± 0.016	0.507	0.695	0.413	0.636
BiLSTM	General + math (seen)	0.667 ± 0.014	0.573	0.833	0.543	0.717
BiLSTM	Code (unseen)	0.528 ± 0.019	0.510	0.659	0.372	0.571

Per family columns are test F1 averaged over seeds 0, 1 and 2.

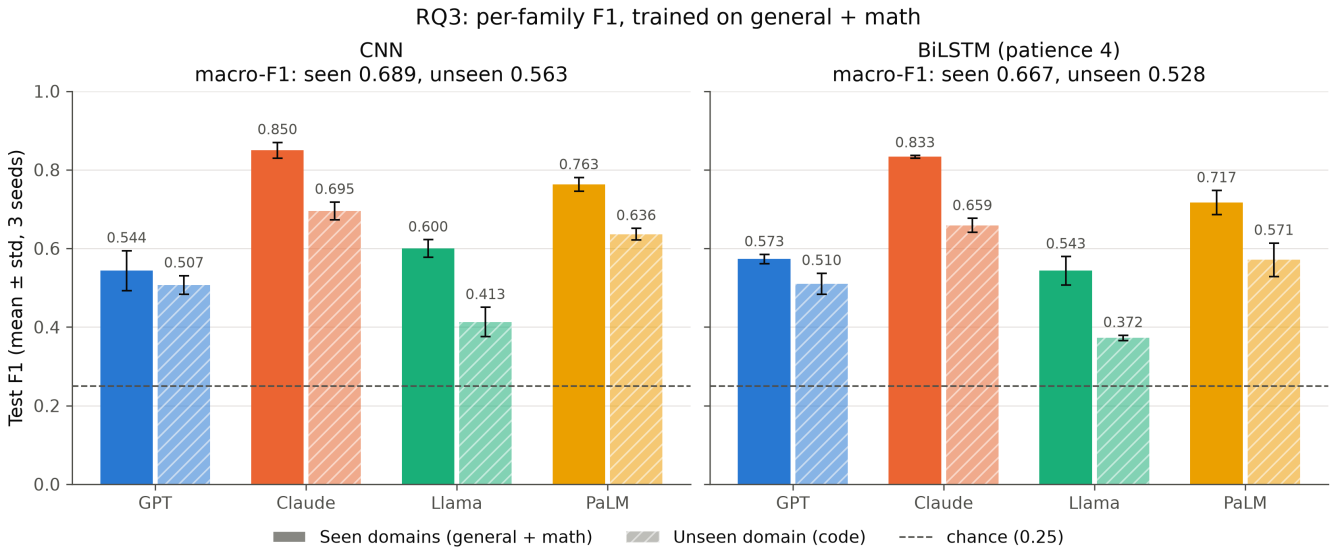


Figure 6. Per family test F1 for models trained on general and math prompts, tested on the same domains (solid) and on unseen code prompts (hatched). Mean ± standard deviation over three seeds.

Does performance decrease on unseen tasks? Yes, by 0.126 macro F1 for the TextCNN and 0.139 for the BiLSTM. Both models still score 0.53 to 0.56 on code, which is more than twice the 0.25 chance level, so a substantial part of the fingerprint carries over to a domain the models never saw.

Are some models more consistently identifiable across domains? The families behave very differently. Llama loses the most for the TextCNN, dropping 0.19, and for the BiLSTM it ties with Claude, both dropping 0.17. Either way Llama ends as the hardest family to identify on code for both models, at 0.41 and 0.37. For the TextCNN that is because it dropped the most, since it started above GPT in domain, and for the BiLSTM it is because it started lower. Claude and PaLM each lose 0.13 to 0.17 but remain the two most identifiable families, so their habits seem to survive the change of task. GPT barely changes, losing only 0.04 and 0.06, but in domain it was already one of the two hardest

families together with Llama, mostly because of their confusion with each other, so there was less domain specific signal for it to lose. One possible explanation for Llama's large drop is that these community fine tunes learned much of their coding style from ChatGPT outputs, so on code their answers look even more like GPT than they do in general conversation. We did not test this directly.

Is the classifier learning a general fingerprint or task specific patterns? Both. The above chance accuracy on unseen code shows a general fingerprint that does not depend on topic, which fits the RQ4 finding that opening and refusal phrases carry much of the signal. The 13 point drop shows that another part of what the models learn is tied to how each family writes the kinds of answers seen in training. Code answers are also partly made of code itself, which leaves less room for the prose habits the classifiers rely on.

Caveats

- The seen and unseen test sets come from different pools of prompts, so the two macro F1 scores are comparable as estimates but were not measured on matched examples. The in domain set is also very slightly unbalanced, with 310 to 318 responses per family.
- The shared vocabulary was built from the full training split, which includes code prompts. Code specific words therefore have embeddings, but those embeddings were never updated by a code example, so this gives the cross domain models little or no advantage.
- Code is a single held out domain. Math was too small (about 160 responses per family) to test on its own.

Error analysis by underlying model

The GPT and Llama confusion is driven mainly by GPT 3.5, not by any single Llama fine tune. To test the explanation from RQ1, we saved every test prediction from the output only models (TextCNN, and BiLSTM with patience 4) and broke the results down by the specific model behind each response.

Model	Family	Test responses	Correct	Labeled GPT	Labeled Llama
gpt-4	GPT	177	65.9%		28.8%
gpt-3.5-turbo	GPT	184	49.5%		33.5%
claude-v1	Claude	234	85.2%	7.7%	5.0%
claude-instant-v1	Claude	127	77.4%	13.1%	7.1%
vicuna-13b	Llama	101	66.7%	20.1%	

Model	Family	Test responses	Correct	Labeled GPT	Labeled Llama
vicuna-7b	Llama	48	65.3%	20.8%	
koala-13b	Llama	90	57.8%	24.8%	
alpaca-13b	Llama	77	60.2%	23.8%	
wizardlm-13b*	Llama	16	54.2%	31.2%	
gpt4all-13b-snoozy*	Llama	14	66.7%	19.0%	
guanaco-33b*	Llama	15	57.8%	22.2%	
palm-2	PaLM	361	72.1%	8.8%	16.1%

TextCNN results, averaged over seeds 0, 1 and 2. Blank cells are the model's own family. * Fewer than 30 test responses, so these rows are too small to interpret.

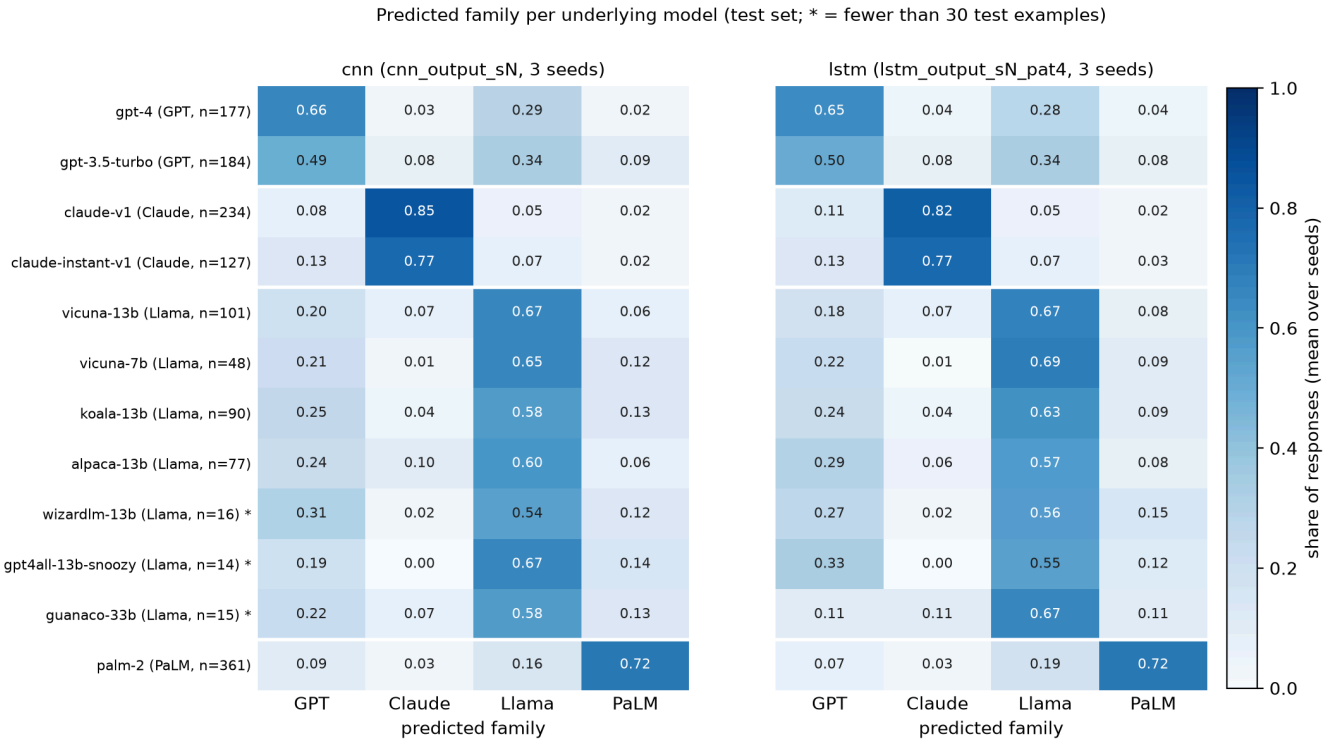


Figure 7. Share of each underlying model's test responses predicted as each family, averaged over three seeds, for the TextCNN (left) and the BiLSTM with patience 4 (right). Asterisks mark models with fewer than 30 test responses.

Vicuna is not special. We initially expected Vicuna, which was trained directly on ChatGPT conversations, to be the main source of the confusion, but its responses are labeled GPT slightly less often (20 to 21%) than Koala's or Alpaca's (24 to 25%). The GPT

likeness is spread across the whole Llama class, which fits the fact that several of these fine tunes were trained on text generated by OpenAI models, not just Vicuna. Our original, more specific explanation was not supported.

GPT 3.5 is the hard case. Within the GPT family, GPT 4 is identified correctly 65.9% of the time, but GPT 3.5 only 49.5% of the time, and a third of GPT 3.5 responses are labeled Llama. GPT 3.5 and its close predecessors are the models whose outputs the Llama fine tunes were trained to imitate, so the model being imitated is the one most often mistaken for its imitators. This is stronger evidence for the imitation explanation than the Vicuna comparison would have been.

Smaller models within a family are harder. Claude Instant is identified less often than Claude v1 (77.4% against 85.2%) and is labeled GPT almost twice as often.

The BiLSTM shows the same pattern. GPT 3.5 is again much harder than GPT 4 (50.4% against 64.8%), and Vicuna is again labeled GPT no more often than Koala or Alpaca (18 to 22% against 24 to 29%).

6. Lessons Learned

The biggest lesson from this project is that the decisions around the models mattered more than the models themselves. The CNN and LSTM ended up tied with a logistic regression on TF-IDF features, while choices about leakage, splitting, early stopping and ablation design changed our conclusions several times.

Data decisions decide what a result means

We inspected the raw data before writing any pipeline code, and that step paid off immediately. The dataset had 20 models, and our first family map missed several LLaMA fine tunes. We also had to decide what counts as leakage. A response that names its model ("I am Vicuna, trained by LMSYS") gives the answer away, so we dropped those responses rather than masking the names, since a mask token would itself become a signal. Habitual phrases like "As an AI" do not name a model, so we kept them, because that kind of phrasing is exactly the fingerprint the project is about. Splitting by question ID mattered just as much, because without it the same prompt could appear in training and test and the input only results would be meaningless. Finally, we learned to split first and balance second, since balancing first leaves the splits unbalanced.

One hyperparameter can change a conclusion

With an early stopping patience of 2, the LSTM looked clearly worse and less stable than the CNN. One seed stopped at epoch 4 with 0.579 macro F1, and with a patience of 4 the same seed reached 0.700, the best LSTM result we had. Instead of dropping the bad seed or quietly changing the setting, we kept the original result as the main one and reported the rerun as a sensitivity check. That changed our answer from "the CNN is better" to "the two are equally accurate, but the CNN is far more robust to training settings."

Always run a simple baseline

A TF-IDF logistic regression scored 0.692, tied with both neural models. If we had not run it, we would have credited the CNN with learning something sophisticated. Instead, the baseline told us the fingerprint lives in short surface patterns, and its top features exposed a cue we had not thought to measure, which is that each family uses a different bullet character.

Test your hypotheses, and expect some to fail

We expected Vicuna, trained on ChatGPT conversations, to explain the confusion between GPT and Llama. The per model error analysis showed Vicuna was no more GPT like than Koala or Alpaca, so that specific idea was wrong. The same analysis found the real pattern instead: GPT 3.5, the model these fine tunes imitated, is the GPT model most often mistaken for Llama. Writing down the failed hypothesis made the final explanation more convincing, not less.

Comparisons hide confounds

Adding the prompt seemed to lower accuracy slightly, until we noticed the combined setting also cut the response from 256 to 192 tokens. A control run showed about half of the drop came from the shorter response. In RQ3, the seen and unseen test sets came from different pools, which we could not avoid but had to state. In both cases the first comparison looked clean, and only checking what else differed between the two settings made the result trustworthy.

Check every claim against every model

Several of our summary statements, such as "GPT is the hardest family" and "Llama drops the most on code," turned out to be true for one model but not the other. They came from reading one table and generalizing. Checking each claim against the result files for both

models caught these before they reached the report.

Ablations are only as good as their implementation

Our markdown stripping ablation required a space after a bullet, so 7% of Claude's • bullets survived it. We only found this because the TF-IDF baseline ranked " • " as Claude's top feature. Similarly, removing the first sentence left 8% of responses unchanged. Neither problem changed our conclusions, but both mean an ablation's effect should be measured on what it actually removed, not on what it was meant to remove.

Engineering lessons

- **Make long jobs resumable and reproducible.** Every run script skipped finished runs, so pausing or crashing never cost more than one run. Fixed seeds made reruns identical, which let us verify results and test the patience explanation exactly.
- **Know your tools.** Python buffers output when it writes to a file, so a healthy training run looked frozen until we checked the process directly.
- **Verify AI assisted work.** We used Claude to plan and draft and Claude Code to write and run the pipeline. This was fast, but it also produced mistakes, including a wrong claim in our own notes about a rerun being identical. Checking every number against the result files caught these, and we would build that check in from the start next time.

What we would do differently

With more time, we would collect our own data by sending the same prompts to current models through their APIs, which would give matched comparisons across families and a larger, more recent dataset. We would also choose early stopping settings by checking their stability across seeds before running the full grid, and we would fine tune a pretrained transformer to see whether contextual models find signals beyond the surface patterns that the CNN, LSTM and TF-IDF models all rely on.

AI Use

Claude AI was used for planning and drafting code.